

DepthVista

DepthVista SDK API Manual



e-con Systems®
Think Vision. Think e-con.

Version 1.3

e-con Systems

5/28/2026

Disclaimer

e-con Systems reserves the right to edit/modify this document without any prior intimation of whatsoever.

Contents

INTRODUCTION TO DEPTHVISTA_HELIX	5
DESCRIPTION	5
PREREQUISITE	5
INSTALLING DEPTHVISTASDK IN LINUX	6
SUPPORTED APIS	7
RESULT INITIALIZE ()	7
RESULT DEINITIALIZE()	7
RESULT GETDEVICECOUNT(UINT32_T* DEVICECOUNT)	8
RESULT GETDEVICEINFO(UINT32_T DEVICEINDEX, DEVICEINFO* DEVICE)	9
RESULT GETDEVICELISTINFO(UINT32_T DEVICECOUNT, DEVICEINFO* DEVICESLIST)	10
RESULT OPENDEVICE(DEVICEINFO DEVINFO, DEVICEHANDLE* DEVHANDLE)	11
RESULT CLOSEDEVICE(DEVICEHANDLE DEVHANDLE)	12
RESULT ISOPENED(DEVICEHANDLE DEVHANDLE)	12
RESULT GETNEXTFRAME(DEVICEHANDLE DEVHANDLE)	13
RESULT GETFRAMES(DEVICEHANDLE DEVHANDLE, FRAMES* FRAMES)	14
INT GETFRAMESPERSECOND(DEVICEHANDLE DEVHANDLE)	16
INT GETTEMPERATURE (DEVICEHANDLE DEVHANDLE, INT* BASETEMPERATURE, INT* LASERTEMPERATURE)	17
RESULT SETDATAMODE(DEVICEHANDLE DEVHANDLE, DATAMODE DATAMODE)	18
RESULT GETDATAMODE(DEVICEHANDLE DEVHANDLE, DATAMODE* DATAMODE)	19
RESULT SETADAPTIVECORING (DEVICEHANDLE DEVHANDLE, INT STATE)	20
RESULT SETTOFIRGAIN(DEVICEHANDLE DEVHANDLE, UINT16_T TOFIRGAIN)	20
RESULT GETTOFIRGAIN(DEVICEHANDLE DEVHANDLE, UINT16_T* TOFIRGAIN)	21
RESULT GETUVCCONTROL(DEVICEHANDLE DEVHANDLE, INT32_T CONTROLID, UVCPROP* CONTROLVALUE)	22
RESULT SETUVCCONTROL(DEVICEHANDLE DEVHANDLE, INT32_T CONTROLID, INT32_T CONTROLVALUE)	23
RESULT SETAVGREGION(DEVICEHANDLE DEVHANDLE, AVGREGION REGION)	25
RESULT SETDEPTHPOS(DEVICEHANDLE DEVHANDLE, DEPTHPTR POS)	25
RESULT SETCURSORCOLOR(DEVICEHANDLE DEVHANDLE, INT COLOR)	26
RESULT SETUNDISTORTION(DEVICEHANDLE DEVHANDLE, INT UNDISTORT)	27
RESULT SETFLYINGPIXELFILTER(DEVICEHANDLE DEVHANDLE, INT STATE)	28
RESULT GETDEPTHIRVALUES(DEVICEHANDLE DEVHANDLE, INT* AVGDEPTH, INT* STDDEPTH, INT* AVGIR, INT* STDIR)	29
VOID REGISTERFRAMECALLBACK(DEVICEHANDLE DEVHANDLE, FUNCTION<VOID(INT, INT)> CB)	30
VOID REGISTERNOTIFICATIONCALLBACK(DEVICEHANDLE DEVHANDLE, FUNCTION<VOID(INT, INT)> CB)	30
RESULT UPDATEAVGXANDY(DEVICEHANDLE DEVHANDLE, INT AVG_X, INT AVG_Y)	31
RESULT UPDATECOLORMAP(DEVICEHANDLE DEVHANDLE, INT MIN, INT MAX, INT COLORMAP)	32
RESULT SETAVGIRDISPLAY(DEVICEHANDLE DEVHANDLE, UINT8_T AVGIRDISPLAY)	32
RESULT CALIBREADREQDEPTHINTRINSIC(DEVICEHANDLE DEVHANDLE, INT* DEPTHINTFILELENGTH)	33

RESULT CALIBREADDEPTHINTRINSIC(DEVICEHANDLE DEVHANDLE, INT* DEPTHINTFILELENGTH, UNSIGNED CHAR* DEPTHINTRINSICBUFFER)	34
RESULT GETDEVICECALIBRATIONPARAMS(DEVICEHANDLE DEVHANDLE, CALIBRATIONPARAMS* CALIBPARAMS)	35
RESULT SETDEPTHNOISE(DEVICEHANDLE DEVHANDLE, UINT16_T VALUE)	36
RESULT GETDEPTHNOISE(DEVICEHANDLE DEVHANDLE, UINT16_T* VALUE)	37
RESULT SETCONFIDENCETHRESHOLD(DEVICEHANDLE DEVHANDLE, UINT16_T VALUE)	37
RESULT GETCONFIDENCETHRESHOLD(DEVICEHANDLE DEVHANDLE, UINT16_T* VALUE)	38
RESULT SETCONFIDENCEMODE(DEVICEHANDLE DEVHANDLE, UINT16_T VALUE)	39
RESULT GETCONFIDENCEMODE(DEVICEHANDLE DEVHANDLE, UINT16_T* VALUE)	40
RESULT GETLASERBOARDTEMPERATUREDATA(DEVICEHANDLE DEVHANDLE, FLOAT *VALUE)	41
RESULT GETBASEBOARDTEMPERATUREDATA(DEVICEHANDLE DEVHANDLE, FLOAT *VALUE)	41
RESULT GETSENSORDIETEMPERATUREDATA(DEVICEHANDLE DEVHANDLE, FLOAT *VALUE)	42
RESULT SETINTEGRATIONTIME(DEVICEHANDLE DEVHANDLE, UINT16_T VALUE)	43
RESULT GETINTEGRATIONTIME(DEVICEHANDLE DEVHANDLE, UINT16_T* VALUE)	44
RESULT SETIMUCONFIG(DEVICEHANDLE DEVHANDLE, IMUCONFIG_TYPEDEF IMUCONFIG)	45
RESULT GETIMUCONFIG(DEVICEHANDLE DEVHANDLE, IMUCONFIG_TYPEDEF* IMUCONFIG)	46
RESULT CONTROLIMUCAPTURE(DEVICEHANDLE DEVHANDLE, IMUDATAINPUT_TYPEDEF* IMUINPUT)	46
RESULT GETIMUVALUE(DEVICEHANDLE DEVHANDLE, IMUDATAINPUT_TYPEDEF* IMUAXES)	47
RESULT GETIMUVALUEGMSL(DEVICEHANDLE DEVHANDLE, IMU_DATA_GMSL* IMUAXES)	48
RESULT READSERIALNO(DEVICEHANDLE DEVHANDLE, INT* SERIALNOLENGTH, UNSIGNED CHAR* SERIALNOBUFFER)	49
RESULT SETLASERSAFETY(DEVICEHANDLE DEVHANDLE, UINT16_T MODE)	50
RESULT GETLASERSAFETY(DEVICEHANDLE DEVHANDLE, UINT16_T* MODE)	51
RESULT LASERRESET(DEVICEHANDLE DEVHANDLE)	52
RESULT SETFPSMODE(DEVICEHANDLE DEVHANDLE, FPSMODE MODE)	52
RESULT GETFPSMODE(DEVICEHANDLE DEVHANDLE, FPSMODE* MODE)	53
ENUM	55
DATAMODE	55
LOGLEVEL	57
LOGGING	57
CAMERAMODE	57
DEPTHRANGE	58
FRAMETYPE	58
FPSMODE	59
RESULT	59
AVGREGION	62
UVCPROPID	62
CALIBVALIDFLAG	63
PIXFORMAT	63
DEVICETYPE	64
STRUCTURE	65

DEVICEINFO	65
INTRINSICCALIBPARAMS	65
EXTRINSICCALIBPARAMS	66
CALIBRATIONPARAMS	66
DEVICEHANDLE	67
UVCPROP	67
DEPTHPTR	68
TOFFFRAME	68
FRAMES	69
IMUCONFIG_TYPEDEF	70
IMUDATAINPUT_TYPEDEF	72
IMUDATAOUTPUT_TYPEDEF	72
IMU_DATA_GMSL	73
 FAQ	 75
 SUPPORT	 76

Introduction to DepthVista_Helix

- DepthVista_GMSL_IP65_ONX is a 3D camera based on Smart Indirect Time of Flight (iTOF) technology, from e-con Systems, a company with over two decades of experience in designing, developing, and manufacturing OEM cameras.
- DepthVista uses a 1/3.2" AF0130 CMOS sensor from Onsemi, designed for precise and high-resolution 3D depth measurement, making it ideal for real-time 3D applications.

Description

- DepthVista is a 3D depth camera based on Onsemi's Smart iTOF sensor, featuring **on-camera depth processing** for accurate 3D measurements without the need for external processing.
- The **1.2 MP** CMOS Smart iToF Sensor, with Advanced 3.5 mm Pixel Stacked BSI Technology offers **Superior Low-Light** and **Ambient-Light Performance**.
- It can provide high-resolution 1.2MP depth output for a range of 0.2m to 2m. With its **Dual Laser operation**, it also delivers **high-accuracy** VGA depth output for a range of **0.5 m to 6m**.

Note: The effective depth range may be reduced under outdoor operating conditions due to ambient sunlight interference. In extreme sunlight environments, the measurable depth range can decrease to as low as 1m in HD mode and 3m in VGA mode.

- Prioritizing safety and reliability, it features Laser **Eye Safety Monitoring**.
- Designed to withstand tough environments, this camera comes with an IP65-rated enclosure, making it durable and versatile.
- It is compatible with NVIDIA Jetson AGX Orin, Orin NX / Nano development kits, enabling easy evaluation and integration into applications.

Prerequisite

The prerequisites are as follows:

- DepthVista console application source code
- CMake (version 3.5 and above)
- DepthVistaSDK
- OpenCV library (version 4.5.4 and above)

Installing DepthVistaSDK in Linux

This section describes the installation of DepthVistaSDK in linux.

The steps to install the DepthVistaSDK are as follows:

1. Run the following command to extract the **package** file.

```
unzip -X <packageName.zip>
```

<Extracted Directory>\Linux\Bin\Ubuntu22.04\ARM\SDK will have an install.sh file.

Note: For Ubuntu 20.04 the install.sh file will be present in <Extracted Directory>\Linux\Bin\Ubuntu20.04\ARM\SDK.

For x64 platform, the install.sh file will be present in <Extracted Directory>\Linux\Bin\Ubuntu20.04\x64 \SDK

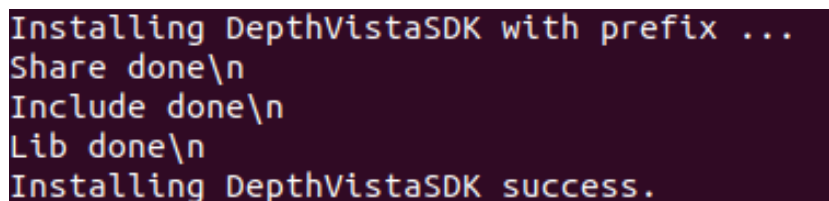
2. Open the folder containing install.sh in terminal.
3. Run the following command to give executable permission for install.sh file.

```
chmod +x install.sh
```

4. Run the following command to install the **DepthVistaSDK**.

```
sudo ./install.sh
```

Once installation is success, **Installation DepthVistaSDK success** message appears.



```
Installing DepthVistaSDK with prefix ...  
Share done\  
Include done\  
Lib done\  
Installing DepthVistaSDK success.
```

Figure 1: Installation success screenshot.

Supported APIs

The details and usage of the supported APIs are explained below.

Result Initialize ()

This function initializes all the needed resources, and it must be called before calling any other APIs in DepthVista SDK.

Parameters	Description
Nil	N/A

Return:

- Returns **Ok** when the device is initialized successfully.
- Returns **NotInitialized** when the device is not initialized successfully.

Sample Code:

```
void Initialize()  
{  
    if (Initialize()>0)  
    {  
        printf("Initialize success");  
    }  
    else  
    {  
        printf("Initialize Failed");  
    }  
}
```

Result Deinitialize()

This function de-initialize all the API in DepthVista SDK and clear all resources allocated. After invoking this API, no other APIs can be invoked.

Parameters	Description
Nil	N/A

Return:

- Returns **Ok** when the device is de-initialized successfully.
- Returns **NotDeInitialized** when the device is not de-initialized.

Sample Code:

```
void Deinitialize()
{
    if(Deinitialize())>0)
    {
        printf("Deinitialize Success\r\n");
    }
    else
    {
        printf("Deinitialize failed\r\n");
    }
}
```

Result GetDeviceCount(uint32_t* deviceCount)

This function is used to get the number of DepthVista devices connected to the host PC. The device count will be stored in the given 32-bit integer pointer.

Parameters	Description
uint32_t* deviceCount	[OUT] Pointer to store device count.

Return:

- Returns **Ok** when the number of devices is obtained successfully.
- Returns **NotInitialized** when the device is not initialized.
- Returns **NoDeviceConnected** when there are no devices connected.

Sample Code:

```
void getDeviceCount()
{
    uint32_t deviceCount;
    if(GetDeviceCount(&deviceCount)>0)
    {
        printf("Number of devices connected is %d\r\n\r\n", deviceCount );
    }
    else
    {
        printf("GetDeviceCount Failed\r\n\r\n");
    }
}
```

Result GetDeviceInfo(uint32_t deviceIndex, DeviceInfo* device)

This function returns the information of the device corresponding to deviceIndex connected to the host PC. The information is contained in a *DeviceInfo* structure. DeviceInfo structure holds the device name, VID, PID, device path and the serial number of the camera.

Parameters	Description
uint32_t deviceIndex	[IN] device index whose device information is to be known.
DeviceInfo * device	[OUT] DeviceInfo struct that contains the information of the device at index deviceIndex.

Member Description:

- char deviceName[50]: This member holds the deviceName of the camera.
- char pid[5]: This member holds the Product ID specific for DepthVista.
- char vid[5]: This member holds the Vendor ID specific for e-con Systems.
- char devicePath[250]: This member holds the path in which the device is enumerated.
- char serialNo[50]: This holds the serial number of the device.

Return:

- Returns **Ok** when the information of all the devices connected was obtained.
- Returns **NotInitialized** when the device is not initialized.
- Returns **NoDeviceConnected** when there are no devices connected.
- Returns **InvalidDeviceIndex** when the device index is invalid.

Sample Code:

```
void GetDeviceInfo_()
{
    uint32_t deviceIndex; // 0 to gDeviceCount -1
    DeviceInfo *gDevicesInfo;
    if(GetDeviceInfo(deviceIndex, gDevicesInfo )>0 )
    {
        printf("GetDeviceInfo Success\r\n");
    }
    else
    {
        printf("GetDevicetInfo failed\r\n");
    }
}
```

Result GetDeviceListInfo(uint32_t deviceCount, DeviceInfo* devicesList)

This function returns the list of information of all DepthVista devices connected to the host PC. The information is contained in a pointer pointing to the array of *DeviceInfo* structure. DeviceInfo structure holds the device name, VID, PID, device path and the serial number of the camera.

Parameters	Description
uint32_t deviceCount	[IN] Number of devices connected.
DeviceInfo* devicesList	[OUT] Array of DeviceInfo whose size is deviceCount.

Return:

- Returns **Ok** when the information of all the devices connected was obtained.
- Returns **NotInitialized** when the device is not initialized.
- Returns **NoDeviceConnected** when there are no devices connected.
- Returns **InvalidDeviceIndex** when the device index is invalid.

Sample Code:

```
void GetDeviceListInfo_()
{
    uint32_t deviceCount; // get device count from
    GetDeviceCount

    DeviceInfo* devicesList = new
    DeviceInfo[deviceCount];

    if(GetDeviceListInfo(deviceCount, devicesList )>0)
    {
        uint16_t index;
        for(index = 0; index < deviceCount; index++)
        {
            printf("Device name is %s",
            devicesList[index].deviceName);

            printf("Device PID is %s",
            devicesList[index].pid);

            printf("Device VID is %s",
            devicesList[index].vid);

            printf("Device devicePath is %s",
            deviceList[index].devicePath);

            printf("Device serialNo is %s",
            deviceList[index].serialNo);

        }
    }
    else
```

```

    {
        printf("GetDeviceListInfo failed\r\n");
    }
}

```

Result OpenDevice(DeviceInfo devInfo, DeviceHandle* devHandle)

This function opens the device specified by devInfo and return a DeviceHandle in the devHandle parameter. This handle grants exclusive access to the device and will be used as parameter in other API calls. The device must be subsequently closed using CloseDevice().

Parameters	Description
DeviceInfo devInfo	[IN] Device information of the device which is to be opened.
DeviceHandle* devHandle	[OUT] Device Handle of the device which has been opened

Return:

- Returns **Ok** when the device corresponding to the devInfo has been opened successfully.
- Returns **InvalidNode** if device has been disconnected but tried to open.
- Returns **CameraAlreadyOpen** if the device is already opened.
- Returns **NotInitialized** when the device is not initialized.
- Returns **NoDeviceConnected** when there are no devices connected.

Sample Code:

```

void OpenDevice_()
{
    DeviceInfo info; //DeviceInfo that we got from
    GetDeviceListInfo()
    DeviceHandle handle;
    if(OpenDevice(info, &handle) >0) //The handle is
    used as input parameter for other APIs
    {
        printf("OpenDevice Success\r\n");
    }
    else
    {
        printf("OpenDevice failed\r\n");
    }
}

```

Result CloseDevice(DeviceHandle devHandle)

This function closes the device that was opened using OpenDevice().

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device which has been closed

Return:

- Returns **Ok** when the device is closed successfully.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```
void CloseDevice_()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(CloseDevice(handle)>0 )
    {
        printf("CloseDevice Success\r\n");
    }
    else
    {
        printf("CloseDevice failed\r\n");
    }
}
```

Result IsOpened(DeviceHandle devHandle)

This function return whether the device corresponding to the given devHandle is opened or not.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device which must checked for open.

Return:

- Returns **Ok** when the device is opened.
- Returns **CameraNotOpened** when the camera is not opened.

Sample Code:

```
void IsOpened_()
{
```

```

    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(IsOpened(handle)>0 )
    {
        printf("Device is opened\r\n");
    }
    else
    {
        printf("Device is closed \r\n");
    }
}

```

Result GetNextFrame(DeviceHandle devHandle)

This function retrieves frame data from the device that is corresponding to the given devHandle. This API must be invoked before retrieving frame data using GetToFFrame().

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.

Return:

- Returns **Ok** when the image frame is retrieved successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **TimeoutError** when the wait event timed out.
- Returns **FrameSeparationFailed** when frame separation fails.

Sample Code:

```

void GetNextFrame_()
{
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(GetNextFrame(handle)>0)
    {
        printf("GetNextFrame success\r\n");
        // call to GetToFFrame();
    }
    else
    {
        printf("GetNextFrame failed\r\n");
    }
}

```

```
}
```

Result GetFrames(DeviceHandle devHandle, Frames* frames)

This function can retrieve the frame data of all *FrameTypes* - RGB, IR, Raw Depth, Depth Color Map, Depth Color Map original and Confidence frame from the device that is corresponding to devHandle. Before invoking this API, invoke GetNextFrame() to receive frame from the device.

Note: Data in Frames structure will be filled based on the current DataMode. For example, if current DataMode is Depth_Mode, only raw_depth frame and depth_colormap will contain data and rest of the frame structure will be a NULL.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
Frames* frames	[OUT] Address of the Frames structure. The structure member are as follows: • ToFFrame rgb : ToFFrame structure with RGB frame data. • ToFFrame ir : ToFFrame structure with IR frame data. • ToFFrame raw_ir: ToFFrame structure with Raw IR frame data. • ToFFrame raw_depth : ToFFrame structure with Raw Depth frame data with post processing. • ToFFrame raw_depth_ordinal: ToFFrame structure with Raw Depth frame data without post processing. • ToFFrame depth : ToFFrame structure with Depth Frame data after post processing. • ToFFrame depth_colormap: ToFFrame structure with DepthColorMap frame data. • ToFFrame Confidence_frame:

TofFrame structure with Confidence frame data.

- **ToFFrame Confidence_frame2:**
TofFrame structure with second Confidence frame data.

Return:

- Returns **Ok** when the Frames structure is retrieved successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **TimeoutError** when the wait event timed out.

Sample Code:

```
void GetFrames_()
{
    Frames frames;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(GetFrames(handle, &frames)>0)
    {
        // Usage of Frames struct
        // RGB frame data
        uint16_t* rgb_image_buf =
(uint16_t*)malloc(frames.rgb.size);
        memcpy(rgb_image_buf, frames.rgb.frame_data,
frames.rgb.size);
        printf("frame width is %d\r\n",
frames.rgb.width);
        printf("frame height is %d\r\n",
frames.rgb.height);

        // IR frame data
        uint16_t* ir_image_buf =
(uint16_t*)malloc(frames.ir.size);
        memcpy(ir_image_buf, frames.ir.frame_data,
frames.ir.size);
        printf("frame width is %d\r\n",
frames.ir.width);
        printf("frame height is %d\r\n",
frames.ir.height);

        //DepthColorMap frame data
```



```

        uint16_t* depth_image_buf =
        (uint16_t*)malloc(frames.depth_colormap.size);

        memcpy(depth_image_buf,
        frames.depth_colormap.frame_data,
        frames.depth_colormap.size);

        printf("frame width is %d\r\n",
        frames.depth_colormap.width);

        printf("frame height is %d\r\n",
        frames.depth_colormap.height);

        //Confidence frame data
        uint16_t* depth_image_buf =
        (uint16_t*)malloc(frames.confidence_frame.size);

        memcpy(depth_image_buf,
        frames.confidence_frame.frame_data,
        frames.confidence_frame.size);

        printf("frame width is %d\r\n",
        frames.confidence_frame.width);

        printf("frame height is %d\r\n",
        frames.confidence_frame.height);
    }
    else
    {
        printf("GetToFFrame failed\r\n");
    }
}

```

Int GetFramesPerSecond(DeviceHandle devHandle)

This function returns the number of frames obtained per second in the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.

Return:

- Returns FPS of the device corresponding to devHandle.
- Returns **CameraNotOpened** when the device is not opened

Sample Code:

```
void GetFramesPerSecond_()
```

```
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    printf("FPS : %d\r\n", getFramesPerSecond(handle));
}
```

Int GetTemperature (DeviceHandle devHandle, int* baseTemperature, int* laserTemperature)

This function returns the number of frames obtained per second in the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int* baseTemperature	[OUT] Base board temperature
int* laserTemperature	[OUT] Laser board temperature

Return:

- Returns **Ok** when temperature values are obtained successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.
- Returns **EmbeddedDataDisabled** when the data embedded in the frame is disabled.

Sample Code:

```
void GetTemperature_()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()

    Int baseBoardTemp, laserBoardTemp;
    GetTemperature(handle, &baseBoardTemp,
    &laserBoardTemp);
    printf("Base Board Temperature : %d\r\n",
    baseBoardTemp);
    printf("Laser Board Temperature : %d\r\n",
    laserBoardTemp);
}
```

Result SetDataMode(DeviceHandle devHandle, DataMode dataMode)

This function sets the stream mode specified by dataMode in the device that is corresponding to the given devHandle. All the supported data modes are listed in the enum *DataMode*.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
DataMode dataMode	[IN]Stream Mode that is to be set. Can we any value from the enum <i>DataMode</i>. The datamodes supported are as follows: • Depth_IR_Conf_Mode: Output depth frame (640 x 480), IR frame (640 x 480) and Confidence frame (640 x 480) at 30 FPS. • Depth_IR_Conf_HD_Mode : Output depth frame (1280 x 960), IR frame (1280 x 960) and Confidence frame (1280 x 960) at 30 FPS.

Return:

- Returns **Ok** when the Data mode is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **SystemCallFail** when the system call API failed.
- Returns **TimeoutError** when the wait event timed out.
- Returns **CameraNotSupported** when the mentioned DataMode is not supported by the device.

Sample Code:

```
void SetDataMode_()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    DataMode dataMode;
    if(SetDataMode(handle, dataMode)>0)
    {
        printf("SetDataMode success\r\n");
    }
    else
    {
        printf("SetDataMode failed\r\n");
    }
}
```

Result GetDataMode(DeviceHandle devHandle, DataMode* dataMode)

This function gets the stream mode in which the device is streaming currently from the device that is corresponding to the given devHandle. All the supported data modes are listed in the enum *DataMode*.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
DataMode* dataMode	[OUT]dataMode is filled with the stream mode in which the device is streaming currently.

Return:

- Returns **Ok** when the Data mode is obtained successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **SystemCallFail** when the system call API failed.

Sample Code:

```
void GetDataMode_()
{
    DataMode* dataMode;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetDataMode(handle, dataMode)>0)
    {
        switch(dataMode)
        {
            case Depth_IR_Conf_HD_Mode:
                printf("Current data modes is
Depth_IR_Conf_HD_Mode \r\n");
            case Depth_IR_Conf_Mode:
                printf("Current data modes is
Depth_IR_Conf_Mode \r\n");
        }
    }
    else
    {
        printf("GetDataMode failed\r\n");
    }
}
```

Result SetAdaptiveCoring (DeviceHandle devHandle, int state)

This function enables or disables adaptive coring in the device that is corresponding to the given devHandle.

Note: Adaptive coring is not supported by DepthVista USB-IRD variant.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int state	[IN]state 1 to enable 0 to disable.

Return Type:

- Returns **Ok** when the TOF Coring value is obtained successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **InvalidDataMode** when trying to call this function in unsupported DataMode.
- Returns **CameraNotSupported** when not supported by the device

Sample API:

```
void SetAdaptiveCoring_()
{
    int state = 1;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetAdaptiveCoring(handle, state)>0)
    {
        printf("SetAdaptiveCoring success\r\n");
    }
    else
    {
        printf("SetAdaptiveCoring failed\r\n");
    }
}
```

Result SetTOFIRGain(DeviceHandle devHandle, uint16_t TOFIRGain)

This function sets the TOF IR gain value specified by TOFIRGain to the device that is corresponding to the given devHandle. The default value is 16.

TOF gain is a software gain added to the IR.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.

`uint16_t TOFIRGain`

[IN]IR Gain value that is to be set. It ranges from 1 to 100.

Return:

- Returns **Ok** when the TOF IR Gain value is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **InvalidValue** when the given value is an invalid one.

Sample Code:

```
void SetTOFIRGain_()
{
    uint16_t TOFIRGain;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetTOFIRGain(handle, TOFIRGain)>0)
    {
        printf("SetTOFIRGain success\r\n");
    }
    else
    {
        printf("SetTOFIRGain failed\r\n");
    }
}
```

Result GetTOFIRGain(DeviceHandle devHandle, uint16_t* TOFIRGain)

This function gets the TOF IR gain value from the device that is corresponding to the given devHandle. The default value is 16.

TOF gain is a software gain added to the IR.

Parameters	Description
<code>DeviceHandle devHandle</code>	[IN] Device Handle of the device.
<code>uint16_t* TOFIRGain</code>	[OUT] TOFIRGain is filled with the TOF IR Gain value. It ranges from 1 to 100.

Return:

- Returns **Ok** when the TOF IR Gain value is obtained successfully.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```
void GetTOFIRGain_()
{
```

```

uint16_t* TOFIRGain;
DeviceHandle handle; //Handle we got from
OpenDevice()
if(GetTOFIRGain(handle, TOFIRGain)>0)
{
    printf("TOF IR Gain value is %d\r\n",
TOFIRGain);
}
else
{
    printf("GetTOFIRGain failed\r\n");
}
}

```

Result GetUVCControl(DeviceHandle devHandle, int32_t controlID, UVCProp* controlValue)

This function gets the UVC control value controlValue of control specified by controlID from the device that is corresponding to the given devHandle. UVC controls supported are as follows:

- Brightness
- Contrast
- Saturation
- Gamma
- Gain
- Sharpness
- White Balance
- Exposure
- Power line frequency

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int32_t controlID	[IN] controlID for the specific UVC Control. Each UVC control have a specific control ID listed in UVCProp enum.
UVCProp* controlValue	[OUT] pointer to the UVCProp structure. UVCProp structure contains ID, minimum value, maximum value, current value, step value and the default value

Return:

- Returns **Ok** when the UVC property is obtained successfully.
- Returns **CameraNotOpened** when the device is not opened.

- Returns **NoPropertyValueGet** when the specified control cannot be obtained.
- Returns **CameraNotSupported** when not supported by the device.

Sample Code:

```
void GetUVCControl_()
{
    int32_t controlId;
    UVCCProp controlValue;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(GetUVCControl(handle, controlId, &controlValue)>0
)
    {
        printf("controlValue.id : %d\r\n",
controlValue.id);
        printf("controlValue.cur : %d\r\n",
controlValue.cur);
        printf("controlValue.min : %d\r\n",
controlValue.min);
        printf("controlValue.max : %d\r\n",
controlValue.max);
        printf("controlValue.step : %d\r\n",
controlValue.step);
        printf("controlValue.default_val : %d\r\n",
controlValue.Default_val);
    }
    else
    {
        printf("GetUVCControl failed\r\n");
    }
}
```

Result SetUVCControl(DeviceHandle devHandle, int32_t controlId, int32_t controlValue)

This function sets the current value of the UVC control specified by controlId to the device that is corresponding to the given devHandle. UVC controls supported by DepthVista are as follows:

- Brightness
- Contrast
- Saturation
- Gamma

- Gain
- Sharpness
- White Balance
- Exposure
- Power line frequency

Parameters	Description
<code>DeviceHandle</code> <code>devHandle</code>	[IN] Device Handle of the device.
<code>int32_t controlID</code>	[IN] controlID for the specific UVC control. Each UVC control have a specific control ID listed UVCProp enum.
<code>int32_t controlValue</code>	[IN] current value that is to be set for the UVC control.

Return Type:

- Returns **Ok** when UVC property is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **NoPropertyValueSet** when the specified control cannot be set.
- Returns **CameraNotSupported** when not supported by the device.
- Returns **CameraNotSupported** when not supported by the device.

Sample Code:

```
void SetUVCControl_()
{
    int32_t controlID;
    int32_t controlValue;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(SetUVCControl(handle, controlID, controlValue)>0
)
    {
        printf("SetUVCControl success\r\n");
    }
    else
    {
        printf("SetUVCControl failed\r\n");
    }
}
```

Result SetAvgRegion(DeviceHandle devHandle, AvgRegion region)

This function sets the average depth and IR calculation area specified by region to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
AvgRegion region	[IN] <i>AvgRegion</i> variable with the region at which the average depth and IR is to be calculated.

- **Center** - Tells that the average depth and IR is calculated at the center of the scene.
- **CustomPtr** – Tells that the average depth and IR is calculated along the mouse pointer.
- **Exit**: Tells to stop average depth and IR calculation.

Return:

- Returns **Ok** when the region at which the average depth and IR is to be calculated is set successfully.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```
void SetAvgRegion_()
{
    AvgRegion region = MouseLivePtr;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetAvgRegion(handle, region)>0)
        printf("SetAvgRegion success\r\n");
    else
        printf("SetAvgRegion failed\r\n");
}
```

Result SetDepthPos(DeviceHandle devHandle, DepthPtr pos)

This function sets the average depth and IR calculation area specified by pos to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
DepthPtr pos	[IN] Object of the structure <i>DepthPtr</i> , containing the x and y co-ordinates.

Return:

- Returns **Ok** when the average Depth and IR calculation area specified by **pos** is success.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```
void SetDepthPos_()
{
    DepthPtr pos;
    pos.X = pos.Y = 100;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetDepthPos(handle, pos)>0)
    {
        printf("SetDepthPos success\r\n");
    }
    else
    {
        printf("SetDepthPos failed\r\n");
    }
}
```

Result SetCursorColor(DeviceHandle devHandle, int color)

This function sets the cursor color to the white or black specified by color parameter to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int color	[IN] Non-Zero value for black cursor color. Zero for white cursor color.

Return:

- Returns **Ok** when the cursor color specified by color parameter is set successfully.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```
void SetCursorColor_()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()
```

```

    if(SetCursorColor(handle, 0)>0)
        printf("Cursor color set to white\r\n");
    else
        printf("SetCursorColor failed\r\n");
}

```

Result SetUnDistortion(DeviceHandle devHandle, int undistort)

This function enables or disables undistortion based on the undistort parameter to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int undistort	[IN] Non-Zero value to enable undistortion. Zero to disable undistortion.

Return:

- Returns **Ok** when undistortion is disabled or enabled based on undistort parameter is success.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **RGB_DCalibNotFound** when RGB_D calibration values are not found.
- Returns **InvalidDataMode** when current data mode does not support the feature.

Sample Code:

```

void SetUnDistortion()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetUnDistortion(handle, 0)>0)
    {
        printf("Undistortion is disbaled\r\n");
    }
    else
    {
        printf("SetUnDistortion failed\r\n");
    }
}

```

Result SetFlyingPixelFilter(DeviceHandle devHandle, int state)

This function enables or disables flying pixel filter based on the state parameter to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int state	[IN] 5 to enable flying pixel filter. 2000 to disable flying pixel filter.

Flying pixels are false pixels which are non-existent but exist in 3D point cloud. The lateral resolution of TOF sensors is generally less. Hence one pixel covers a large area in 3D space. If this area has objects at different depths, they combine into a single pixel causing flying pixels. This effect is more prominent at the boundaries of the objects. This is because at the edges, light from foreground and background may combine to produce intermediate erroneous depth values, which can be seen as a wave of pixels known as flying pixels. If a pixel has large depth discontinuity from its neighboring pixels, it is considered as flying pixel. This pixel will be termed as invalid and ignored in 3D point cloud.

Return:

- Returns **Ok** when flying pixel filter is disabled or enabled based on **state** parameter is success.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **InvalidDataMode** when current data mode does not support the feature.

Sample Code:

```
void SetFlyingPixelFilter_()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetFlyingPixelFilter(handle, 0)>0)
    {
        printf("SetFlyingPixelFilter is
disbaled\r\n");
    }
    else
    {
        printf("SetFlyingPixelFilter failed\r\n");
    }
}
```

Result GetDepthIRValues(DeviceHandle devHandle, int* avgDepth, int* stdDepth, int* avgIR, int* stdIR)

This function gets the average depth and IR and sigma depth and IR from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int* avgDepth	[OUT] pointer to int variable that is to be filled with the average depth value.
int* stdDepth	[OUT] pointer to int variable that is to be filled with the sigma depth value.
int* avgIR	[OUT] pointer to int variable that is to be filled with the average IR value.
int* stdIR	[OUT] pointer to int variable that is to be filled with the sigma IR value.

Return:

- Returns **Ok** when average depth and IR, and sigma depth and IR is obtained successfully.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```
void GetDepthIRValues_()
{
    int avgDepth, stdDepth, avgIR, stdIR;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetDepthIRValues(handle, &avgDepth, &stdDepth,
    &avgIR, &stdIR)>0)
    {
        printf("Average depth value is %d\r\n",
        avgDepth);
        printf("sigma depth value is %d\r\n",
        stdDepth);
        printf("Average IR value is %d\r\n", avgIR);
        printf("sigma IR value is %d\r\n", stdIR);
    }
    else
    {
        printf("GetDepthIRValues failed\r\n");
    }
}
```

```

    }
}

```

Void RegisterFrameCallback(DeviceHandle devHandle, function<void(int, int)> cb)

Register a callback function specified by cb when frame is received from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
function<void(int, int)> cb	[IN] Function pointer of the callback function.

Sample Code

```

void callback_func(int a, int b)
{
    printf("Frames received from device %d\r\n, b");
}

void RegisterFrameCallback_()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    RegisterFrameCallback(handle, &callback_func);
}

```

Void RegisterNotificationCallback(DeviceHandle devHandle, function<void(int, int)> cb)

This function registers a callback function specified by cb when notification is received from the device that is corresponding to the given devHandle. This is used to receive device removal notification.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
function<void(int, int)> cb	[IN] Function pointer of the callback function.

Sample Code :

```

void callback_func(int a, int b)
{

```

```

        printf("Device removed notification received from
device %d \r\n", b);
    }
void RegisterNotificationCallback_()
{
    DeviceHandle handle; //Handle we got from
OpenDevice()
    RegisterNotificationCallback(handle,
&callback_func);
}

```

Result UpdateAvgXandY(DeviceHandle devHandle, int avg_x, int avg_y)

This function updates the ROI for which the average depth and IR is being calculated of the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int avg_x	[IN] width of the average depth calculating ROI
int avg_y	[IN] height of the average depth calculating ROI

Return:

- Returns **Ok** when the ROI for average depth and IR is set successfully.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```

void UpdateAvgXandY_()
{
    int avg_x = 64, avg_y = 64;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(UpdateAvgXandY(handle, avg_x, avg_y)>0)
    {
        printf("UpdateAvgXandY success\r\n");
    }
    else
    {
        printf("UpdateAvgXandY failed\r\n");
    }
}

```


Result UpdateColorMap(DeviceHandle devHandle, int min, int max, int colormap)

This function updates the colormap that is being applied on the raw depth data to the device that is corresponding to the given devHandle, along with the minimum and maximum limit of depth data to be covered in the depth colormap scene.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int min	[IN] minimum limit of depth data that is to be included in the colormap image.
int max	[IN] maximum limit of depth data that is to be included in the colormap image.
int colormap	[IN] specific number for a colormap. There are 12 colormap and the values are from 0 to 11.

Return:

- Returns **Ok** when the minimum and maximum limit of depth, and the colormap is updated successfully.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```
void UpdateColorMap_()
{
    int min = 200, max = 1200, colormap = 4;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(UpdateColorMap(handle, min, max, colormap)>0)
    {
        printf("UpdateColorMap success\r\n");
    }
    else
    {
        printf("UpdateColorMap failed\r\n");
    }
}
```

Result SetAvgIRDisplay(DeviceHandle devHandle, uint8_t avgIRDisplay)

This function sets whether to display the IR data along with the average Depth Value.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint8_t avgIRDisplay	[IN] uint8_t variable to set or unset the IR display along with Average depth. • 0 to unset • 1 to set

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.

Sample Code:

```
void SetAvgIRDisplay_()
{
    int avgIRDisplay = 0;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetAvgIRDisplay(handle, avgIRDisplay)>0)
    {
        printf("SetAvgIRDisplay success\r\n");
    }
    else
    {
        printf("SetAvgIRDisplay failed\r\n");
    }
}
```

Result CalibReadReqDepthIntrinsic(DeviceHandle devHandle, int* depthIntFileLength)

This function requests the device to read intrinsic calibration data of the depth camera that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int* depthIntFileLength	[OUT] Pointer to int data that contains the length of the valid calibration Data on success.

Return:

- Returns **Ok** when the request to read intrinsic calibration data of the depth camera is success.

- Returns **CameraNotOpened** when the device is not opened.
- Returns **NoPropertyValueGet** when the specified control cannot be obtained.
- Returns **HidWriteFail** when the buffer cannot be written to the hid handle.
- Returns **HidReadFail** when the buffer cannot be read from the hid handle.
- Returns **TimeoutError** when the wait event timed out.

Sample Code:

```
void calibReadReqDepthIntrinsic_()
{
    int depthIntFileLength;
    DeviceHandle handle; //Handle we got from
    OpenDevice()

    if(CalibReadReqDepthIntrinsic(handle,
    &depthIntFileLength)>0)
    {
        printf("CalibReadReqDepthIntrinsic
    success\r\n");
    }
    else
    {
        printf("CalibReadReqDepthIntrinsic
    failed\r\n");
    }
}
```

Result CalibReadDepthIntrinsic(DeviceHandle devHandle, int* depthIntFileLength, unsigned char* DepthIntrinsicBuffer)

This function reads the intrinsic calibration data of the depth camera that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int* depthIntFileLength	[OUT] Pointer to int data that contains the length of the valid calibration Data that is read.
unsigned char* DepthIntrinsicBuffer	[OUT] Pointer to an unsigned char that will contain the intrinsic calibration Data of depth camera on success of the API.

Return:

- Returns **Ok** when the intrinsic calibration data of the depth camera is read successfully.

- Returns **CameraNotOpened** when the device is not opened.
- Returns **NoPropertyValueGet** when the specified control cannot be obtained.
- Returns **HidWriteFail** when the buffer cannot be written to the hid handle.
- Returns **HidReadFail** when the buffer cannot be read from the hid handle.
- Returns **TimeoutError** when the wait event timed out.

Sample Code:

```
void calibReadDepthIntrinsic_()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()

    int depthIntFileLength;
    unsigned char* DepthIntrinsicBuffer;
    int read_length_depth; //read_length_data must be
    the length requested using calibReadReqDepthIntrinsic API
    DepthIntrinsicBuffer = (unsigned
    char*)malloc(read_length_depth);
    if(CalibReadDepthIntrinsic(handle,
    &depthIntFileLength, DepthIntrinsicBuffer)>0)
    {
        printf("CalibReadDepthIntrinsic success\r\n");
    }
    else
    {
        printf("CalibReadDepthIntrinsic failed\r\n");
    }
    free(DepthIntrinsicBuffer);
}
```

Result GetDeviceCalibrationParams(DeviceHandle devHandle, CalibrationParams* calibparams)

This function gets the Calibration parameters of the device that is corresponding to the given devHandle and fills the values in calibparams

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
CalibrationParams* calibparams	[OUT] pointer to CalibrationParams structure to store the intrinsic and extrinsic calibration parameters.

Return:

- Returns **Ok** when CalibrationParams Data read is successfully.
- Returns **RGB_DCalibNotFound** when RGB_D calibration values are not found.

Sample Code:

```
void GetDeviceCalibrationParams_()
{
    DeviceHandle handle; //Handle we got from
OpenDevice()
    CalibrationParams calibparams;
    if(GetDeviceCalibrationParams(handle,
&calibparams)>0)
    {
        printf("Get Calibration parameters
success\r\n");
    }
    else
    {
        printf("Get Calibration parameters
failed\r\n");
    }
}
```

Result SetDepthNoise(DeviceHandle devHandle, uint16_t value)

This function sets Depth Noise value to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t value	[IN] value that is to be set. It ranges from 0 to 15.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void SetDepthNoise _()
{
    uint16_t value;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(SetDepthNoise(handle, value)>0)
```

```

{
    printf("SetDepthNoise success\r\n");
}
else
{
    printf("SetDepthNoise failed\r\n");
}

```

Result GetDepthNoise(DeviceHandle devHandle, uint16_t* value)

This function Gets Depth Noise value from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t* value	[OUT] uint16_t variable to get depth noise

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```

void GetDepthNoise _()
{
    uint16_t value;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetDepthNoise (handle, &value)>0)
    {
        printf("GetDepthNoise success\r\n");
    }
    else
    {
        printf("GetDepthNoise failed\r\n");
    }
}

```

Result SetConfidenceThreshold(DeviceHandle devHandle, uint16_t value)

This function sets Confidence threshold value to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t value	[IN] uint16_t variable to set confidence threshold. It ranges from 0 to 4095.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void SetConfidenceThreshold _()
{
    uint16_t value;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetConfidenceThreshold (handle, value)>0)
    {
        printf("SetConfidenceThreshold success\r\n");
    }
    else
    {
        printf("SetConfidenceThreshold failed\r\n");
    }
}
```

Result GetConfidenceThreshold(DeviceHandle devHandle, uint16_t* value)

This function Gets Confidence threshold value from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t* value	[OUT] uint16_t variable to get confidence threshold value.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void GetConfidenceThreshold _()
```

```

{
    uint16_t value;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetConfidenceThreshold (handle, &value)>0)
    {
        printf("GetConfidenceThreshold success\r\n");
    }
    else
    {
        printf("GetConfidenceThreshold failed\r\n");
    }
}
}
}

```

Result SetConfidenceMode(DeviceHandle devHandle, uint16_t value)

This function used to set the confidence frame mode to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t value	[IN] value to set confidence mode. 0 for Amplitude, 4 for SNR.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```

void SetConfidenceMode _()
{
    uint16_t value;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetConfidenceMode (handle, value)>0)
    {
        printf("SetConfidenceMode success\r\n");
    }
}

```



```

else
{
    printf("SetConfidenceMode failed\r\n");
}
}
}
}

```

Result GetConfidenceMode(DeviceHandle devHandle, uint16_t *value)

This function used to get the confidence frame mode from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t *value	[OUT] value to get confidence mode. 0 for Amplitude, 4 for SNR.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```

void GetConfidenceMode _()
{
    Uint16_t value;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(GetConfidenceMode (handle, &value)>0)
    {
        printf("GetConfidenceMode success\r\n");
    }
    else
    {
        printf("GetConfidenceMode failed\r\n");
    }
}
}
}

```

Result GetLaserBoardTemperatureData(DeviceHandle devHandle, float *value)

This function used to get the laser board temperature from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
float *value	[OUT] value to get the laser board temperature data.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void GetLaserBoardTemperatureData _()
{
    float value;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if (GetLaserBoardTemperatureData (handle, &value)>0)
    {
        printf("GetLaserBoardTemperatureData
        success\r\n");
    }
    else
    {
        printf("GetLaserBoardTemperatureData
        failed\r\n");
    }
}
```

Result GetBaseBoardTemperatureData(DeviceHandle devHandle, float *value)

This function used to get the base board temperature from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
float *value	[OUT] value to get the base board temperature data.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void GetBaseBoardTemperatureData _()
{
    float value;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetBaseBoardTemperatureData (handle, &value)>0)
    {
        printf("GetBaseBoardTemperatureData
        success\r\n");
    }
    else
    {
        printf("GetBaseBoardTemperatureData
        failed\r\n");
    }
}
}
```

Result GetSensorDieTemperatureData(DeviceHandle devHandle, float *value)

This function used to get the sensor die temperature from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
float *value	[OUT] value to get the sensor die temperature data.

Return:

- Returns **Ok** when the display IR feature is set successfully.

- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void GetSensorDieTemperatureData _()
{
    float value;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetSensorDieTemperatureData (handle, &value)>0)
    {
        printf("GetSensorDieTemperatureData
success\r\n");
    }
    else
    {
        printf("GetSensorDieTemperatureData
failed\r\n");
    }
}
}
```

Result SetIntegrationTime (DeviceHandle devHandle, uint16_t value)

This function sets Integration time to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t value	[IN] uint16_t variable to set integration time. It ranges from 0 to 850.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void SetConfidenceThreshold _()
{
    uint16_t value;
```

```

    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(SetIntegrationTime (handle, value)>0)
    {
        printf("SetIntegrationTimesuccess\r\n");
    }
    else
    {
        printf("SetIntegrationTimefailed\r\n");
    }

```

Result GetIntegrationTime (DeviceHandle devHandle, uint16_t* value)

This function gets integration time from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t* value	[OUT] uint16_t variable to get integration time value.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```

void GetIntegrationTime_()
{
    uint16_t value;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(GetIntegrationTime (handle, &value)>0)
    {
        printf("GetIntegrationTimesuccess\r\n");
    }
    else
    {
        printf("GetIntegrationTimefailed\r\n");
    }
}

```

```
}  
}
```

Result SetIMUConfig(DeviceHandle devHandle, IMUCONFIG_TypeDef IMUConfig)

This function sets the IMU Configuration mode of the IMU in the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
IMUCONFIG_TypeDef IMUConfig	[IN] pointer to the IMUCONFIG_TypeDef structure.

Return:

- Returns **Ok** when the IMU Configuration mode is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **NoPropertyValueSet** when the specified control cannot be set.
- Returns **SystemCallFail** when the system call API failed.
- Returns **HidWriteFail** when the buffer cannot be written to the hid handle.
- Returns **HidReadFail** when the buffer cannot be read from the hid handle.
- Returns **TimeoutError** when the wait event timed out.

Sample Code:

```
void SetIMUConfig_()
{
    IMUCONFIG_TypeDef IMUConfig;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    //Set the IMUConfig to the required Configuration
    if(SetIMUConfig(handle, IMUConfig)>0 )
    {
        printf("SetIMUConfig success\r\n");
    }
    else
    {
        printf("SetIMUConfig Failed\r\n");
    }
}
```

Result GetIMUConfig(DeviceHandle devHandle, IMUCONFIG_TypeDef* IMUConfig)

This function gets the IMU Configuration mode of the IMU from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
IMUCONFIG_TypeDef* IMUConfig	[OUT] pointer to the <u>IMUCONFIG_TypeDef</u> structure.

Return:

- Returns **Ok** when the IMU Configuration mode is obtained successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **NoPropertyValueGet** when the specified control cannot be obtained.
- Returns **SystemCallFail** when the system call API failed.
- Returns **HidWriteFail** when the buffer cannot be written to the hid handle.
- Returns **HidReadFail** when the buffer cannot be read from the hid handle.
- Returns **TimeoutError** when the wait event timed out.

Sample Code:

```
void GetIMUConfig_()
{
    IMUCONFIG_TypeDef IMUConfig;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetIMUConfig(handle, &IMUConfig)>0 )
    {
        printf("GetIMUConfig success\r\n");
    }
    else
    {
        printf("GetIMUConfig Failed\r\n");
    }
}
```

Result ControlIMUCapture(DeviceHandle devHandle, IMUDATAINPUT_TypeDef* IMUInput)

This function sets the IMU Value update in the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
IMUDATAINPUT_TypeDef* IMUInput	[IN] pointer to the <u>IMUDATAINPUT_TypeDef</u> structure.

Return:

- Returns **Ok** when the IMU Value update is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **NoPropertyValueSet** when the specified control cannot set.
- Returns **SystemCallFail** when the system call API failed.
- Returns **HidWriteFail** when the buffer cannot be written to the hid handle.
- Returns **HidReadFail** when the buffer cannot be read from the hid handle.
- Returns **TimeoutError** when the wait event timed out.

Sample Code:

```
void ControlIMUCapture_()
{
    IMUDATAINPUT_TypeDef IMUInput;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    //Set the IMUInput to the required Mode
    if(ControlIMUCapture(handle, &IMUInput)>0 )
    {
        printf("ControlIMUCapture success\r\n");
    }
    else
    {
        printf("ControlIMUCapture Failed\r\n");
    }
}
```

Result GetIMUValue(DeviceHandle devHandle, IMUDATAINPUT_Typedef* IMUAxes)

This function gets the IMU axis data from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
IMUDATAINPUT_Typedef* IMUAxes	[OUT] pointer to the <u>IMUDATAINPUT_Typedef</u> structure.

Return:

- Returns **Ok** when the IMU axis data is received successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **SystemCallFail** when the system call API failed.
- Returns **HidWriteFail** when the buffer cannot be written to the hid handle.
- Returns **HidReadFail** when the buffer cannot be read from the hid handle.
- Returns **TimeoutError** when the wait event timed out.
- Returns **Failed** when get operation fails.

Sample Code:

```
void GetIMUValue_()
{
    IMUDATAINPUT_Typedef IMUAxes;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetIMUValue(handle, &IMUAxes)>0 )
    {
        printf("GetIMUValue success\r\n");
    }
    else
    {
        printf("GetIMUValue Failed\r\n");
    }
}
```

Result GetIMUValueGMSL(DeviceHandle devHandle, IMU_DATA_GMSL* IMUAxes)

This function gets the IMU axis data from the GMSL device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
IMU_DATA_GMSL* IMUAxes	[OUT] pointer to the IMU_DATA_GMSL structure.

Return:

- Returns **Ok** when the IMU axis data is received successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **SystemCallFail** when the system call API failed.
- Returns **HidWriteFail** when the buffer cannot be written to the hid handle.

- Returns **HidReadFail** when the buffer cannot be read from the hid handle.
- Returns **TimeoutError** when the wait event timed out.
- Returns **Failed** when get operation fails.

```
void GetIMUValueGMSL_()
{
    IMU_DATA_GMSL IMUAxes;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(GetIMUValueGMSL(handle, &IMUAxes)>0 )
    {
        printf("GetIMUValueGMSL success\r\n");
    }
    else
    {
        printf("GetIMUValueGMSL Failed\r\n");
    }
}
```

Result ReadSerialNo(DeviceHandle devHandle, int* serialNoLength, unsigned char* serialNoBuffer)

This function Reads the serial number from device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
int* serialNoLength	[OUT] pointer to the integer value which holds the serial No length.
Unsigned char* serialNoBuffer	[OUT] pointer to the unsigned character array which holds the serial No buffer.

Return:

- Returns **Ok** when the IMU axis data is received successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **SysCallFail** when the system call API failed.
- Returns **NoPropertyValueGet** when the specified control cannot be obtained.ss
- Returns **TimeoutError** when the wait event timed out.
- Returns **Failed** when get operation fails.

```
void ReadSerialNo_()
{
    int serialNoLength;
    std::vector<unsigned char>
serialNoBuffer(SERIAL_NUM_LENGTH, 0);
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(GetIMUValueGMSL(handle, &serialNoLength,
serialNoBuffer.data() )>0 )
    {
        printf("ReadSerialNo success\r\n");
    }
    else
    {
        printf("ReadSerialNo Failed\r\n");
    }
}
```

Result SetLaserSafety (DeviceHandle devHandle, uint16_t mode)

This function sets laser safety to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t mode	[IN] uint16_t variable to set mode of the laser safety. 0 -> Laser safety OFF 1 -> Laser safety ON

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.
- Returns **SysCallFail** when the system call API failed.

Sample Code:

```
void SetLaserSafety _()
{
    uint16_t mode;
```

```

    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(SetLaserSafety (handle, mode )>0)
    {
        printf("SetLaserSafety success\r\n");
    }
    else
    {
        printf("SetLaserSafety failed\r\n");
    }
}

```

Result GetLaserSafety (DeviceHandle devHandle, uint16_t* mode)

This function gets the laser safety from the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.
uint16_t* mode	[OUT] uint16_t variable to get the current laser safety mode. 0 -> Laser safety OFF. 1 -> Laser safety ON.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.
- Returns **SysCallFail** when the system call API failed.

Sample Code:

```

void GetLaserSafety_()
{
    uint16_t mode;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(GetLaserSafety (handle, &mode)>0)
    {
        printf("GetLaserSafety success\r\n");
    }
    else
    {

```

```
printf("GetLaserSafety      failed\r\n");
    }
}
```

Result LaserReset (DeviceHandle devHandle)

This function sets laser reset to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.
- Returns **SysCallFail** when the system call API failed.

Sample Code:

```
void LaserReset _()
{
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(LaserReset (handle)>0)
    {
        printf("Laser Reset success\r\n");
    }
    else
    {
        printf("Laser Reset failed\r\n");
    }
}
```

Result SetFPSMode (DeviceHandle devHandle, FpsMode mode)

This function sets FPS Mode to the device that is corresponding to the given devHandle.

Parameters	Description
DeviceHandle devHandle	[IN] Device Handle of the device.

`FpsMode mode`

[IN] FpsMode variable to set FPS Mode.
 0 -> TOF_30Fps
 1 -> TOF_50Fps

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void SetFPSMode _()
{
    FpsMode mode;
    DeviceHandle handle; //Handle we got from
    OpenDevice()
    if(SetFPSMode (handle, mode )>0)
    {
        printf("SetFPSMode success\r\n");
    }
    else
    {
        printf("SetFPSMode failed\r\n");
    }
}
```

Result GetFPSMode (DeviceHandle devHandle, FpsMode* mode)

This function gets the laser safety from the device that is corresponding to the given devHandle.

Parameters	Description
<code>DeviceHandle devHandle</code>	[IN] Device Handle of the device.
<code>FpsMode* mode</code>	[OUT] FpsMode variable to get the current FPS mode. 0 -> TOF_30Fps. 1 -> TOF_50Fps.

Return:

- Returns **Ok** when the display IR feature is set successfully.
- Returns **CameraNotOpened** when the device is not opened.
- Returns **CameraNotSupported** when the called using an unsupported device.

Sample Code:

```
void GetFPSMode_()
{
    FpsMode mode;
    DeviceHandle handle; //Handle we got from
OpenDevice()
    if(GetFPSMode (handle, &mode)>0)
    {
        printf("GetFPSMode success\r\n");
    }
    else
    {
        printf("GetFPSMode failed\r\n");
    }
}
```

Enum

The details regarding the enum used will be discussed below.

DataMode

Different datamode supported by DepthVista is listed in the DataMode enum.

```
typedef enum {  
    Depth_IR_Mode = 0,  
    Depth_IR_Conf_Mode = 1,  
    Depth_Mode = 2,  
    IR_Mode = 3,  
    Depth_IR_RGB_VGA_Mode = 4,  
    Depth_IR_RGB_HD_Mode = 5,  
    RGB_VGA_Mode = 6,  
    RGB_HD_Mode = 7,  
    RGB_Full_HD_Mode = 8,  
    RGB_1200p_Mode = 9,  
    Depth_IR_C1_C2_Mode = 10,  
    Depth_RGB_HD_Mode = 11,  
    Depth_IR_C1_RGB_HD_Mode = 12,  
    Depth_IR_C1_C2_RGB_HD_Mode = 13,  
    RGB_960x540_Mode_100fps = 14,  
    RGB_Full_HD_Mode_60fps = 15,  
    RGB_4K_Mode_20fps = 16,  
    RGB_4K_Mode_30fps = 17,  
    IR_RGB_HD_Mode = 18,  
    RGB_4K_Mode_15fps = 19,  
    Depth_IR_Conf_HD_Mode = 20,  
}DataMode;
```

Where,

Depth_IR_Mode: Output depth frame (640 x 480) and IR frame (640 x 480) at 30 FPS.

Depth_IR_Conf_Mode: Output depth frame (640 x 480), IR frame (640 x 480) and Confidence frame (640 x 480) at 30 FPS.

Depth_Mode: Output depth frame (640 x 480) at 30 FPS.

IR_Mode: Output IR frame (640 x 480) at 30 FPS.

Depth_IR_RGB_VGA_Mode: Output depth frame (640 x 480), IR frame (640 x 480) and RGB frame (640 x 480) at 15 FPS.

Depth_IR_RGB_HD_Mode: Output depth frame (640 x 480), IR frame (640 x 480) and RGB frame (1280 x 720) at 15 FPS.

RGB_VGA_Mode: Output RGB frame (640 x 480) at 60 FPS.

RGB_HD_Mode: Output RGB frame (1280 x 720) at 60 FPS.

RGB_Full_HD_Mode: Output RGB frame (1920 x 1080) at 30 FPS.

RGB_1200p_Mode: Output RGB frame (1920 x 1200) at 30 FPS.

Depth_IR_C1_C2_Mode: Output depth frame (640 x 480), IR frame (640 x 480), Confidence frame 1 (640 x 480) and Confidence frame 2 (640 x 480) at 30 FPS.

Depth_RGB_HD_Mode: Output depth frame (640 x 480) and RGB frame (1280 x 720) at 30 FPS.

Depth_IR_C1_RGB_HD_Mode: Output depth frame (640 x 480), IR frame (640 x 480), Confidence frame (640 x 480), RGB frame (1280 x 720) at 30 FPS.

Depth_IR_C1_C2_RGB_HD_Mode: Output depth frame (640 x 480), IR frame (640 x 480), Confidence frame 1 (640 x 480), Confidence frame 2 (640 x 480), RGB frame (1280 x 720) at 30 fps.

RGB_960x540_Mode_100fps: Output RGB frame (960 x 540) at 100 FPS.

RGB_Full_HD_Mode_60fps: Output RGB frame (1920 x 1080) at 60 FPS.

RGB_4K_Mode_20fps: Output RGB frame (3840 x 2160) at 20 FPS.

RGB_4K_Mode_30fps: Output RGB frame (3840 x 2160) at 30 FPS.

IR_RGB_HD_Mode: Output IR frame (640 x 480), RGB frame (1920 x 1080) at 30 fps.

RGB_4K_Mode_15fps: Output RGB frame (3840 x 2160) at 15 FPS.

Depth_IR_Conf_Mode: Output depth frame (1920 x 1080), IR frame (1920 x 1080) and Confidence frame 1920 x 1080) at 30 FPS.

LogLevel

LogLevel enum contains the level of the log that is to be stored in log file or to print in console.

```
typedef enum{  
    Low = 0,  
    Critical = 1,  
    High = 2,  
    Disable = 3  
}LogLevel;
```

Where,

Low: Enable All logs.

Critical: Enable very critical logs.

High: Enables high priority logs.

Disable: Enable all logs.

Logging

Logging enum contains the options for logging debug prints.

```
typedef enum{  
    Console = 1,  
    LogFile = 2,  
    BothConsoleAndLogFile = 3,  
}Logging;
```

Where,

Console: Prints the log only in Console.

LogFile: Prints the log only in Log File.

BothConsoleAndLogFile: Prints the log in both Console and Log File.

CameraMode

Different camera mode supported by DepthVista is listed in the CameraMode enum

```
typedef enum {  
    DualCameraMode = 0,
```

```
    RGBCameraMode = 1,  
    TOFCameraMode = 2,  
} CameraMode;
```

Where,

DualCameraMode: Both RGB and TOF camera enabled mode.

RGBCameraMode: Only RGB camera enabled mode.

TOFCameraMode: Only TOF camera enabled mode.

DepthRange

Different depth range supported by DepthVista is listed in the DepthRange enum

```
typedef enum {  
    NearRange = 0,  
    WDRRange = 1,  
    FarRange = 1,  
} DepthRange;
```

Where,

NearRange: Near range mode.

WDRRange: WDR range mode.

FarRange: Far range mode.

FrameType

Types of frames that can be obtained from DepthVista is listed in the FrameType enum.

```
typedef enum {  
    IRPreviewFrame,  
    DepthColorMap,  
    DepthColorMapOriginal,  
    RGBFrame,  
    DepthRawFrame,  
    ConfidenceFrame,  
} FrameType;
```

Where,

IRPreviewFrame: Separate IR frame.

DepthColorMap: Depth data that is applied with Color map for preview purpose.

DepthColorMapOriginal: Raw Depth data without any filters that is applied with Color map for preview purpose.

RGBFrame: Separate RGB frame.

DepthRawFrame: Raw depth frame without applying color map. Hence this frame cannot be used for preview.

ConfidenceFrame: Raw confidence frame. This frame cannot be used for preview.

FpsMode

Different FpsMode supported by DepthVista HELIX is listed in the FpsMode enum

```
typedef enum {  
    TOF_30Fps = 0,  
    TOF_50Fps = 1,  
} FpsMode;
```

Where,

TOF_30Fps: The FPS will be 30.

TOF_50Fps: The FPS will be 50.

Result

Result enum contain the values that will be returned from supported APIs.

```
typedef enum {  
    Ok = 1,  
    NotInitialized = -1,  
    NotDeInitialized = -2,  
    InvalidFrameType = -3,  
    NoStreaming = -4,  
    AlreadyStreaming = -5,  
    InvalidNode = -6,  
    CameraNotOpened = -7,  
    InvalidDeviceIndex = -8,  
    NoDeviceConnected = -9,  
    NoPropertyValueGet = -10,  
    NoPropertyValueSet = -11,  
    SystemCallFail = -12,
```

```

InvalidValue = -13,
HidWriteFail = -14,
HidReadFail = -15,
UVCOpenFail = -16,
TimeoutError = -17,
InvalidBuffer = -18,
RGB_DCalibNotFound = -19,
FrameSeparationFailed = -20,
RegisterCallBackFailed = -21,
CameraAlreadyOpen = -22,
InvalidDataMode = -23,
Failed = -24,
ProcessPtrOpenFailed = -25,
ProcessPtrCloseFailed = -26,
NoGraphicCardFound = -27,
InvalidDepthMode = -28,
Others = -255,
ProcessPtrOpenFailed = -25,
ProcessPtrCloseFailed = -26,
NoGraphicCardFound = -27,
InvalidDepthMode = -28,
CameraNotSupported = -29,
SizeQueryFailed = -30,
FrameQueryFailed = -31,
FilterNotApplied = -32,
TOFCalibrationNotFound = -33,
DeviceNotCapable = -34,
}Result;

```

Where,

Ok: The API completed successfully.

NotInitialized: The APIs are not initialized.

NotDeInitialized: The APIs are not deinitialized.

InvalidFrameType: The input frame type is invalid.

NoStreaming: The camera is not streaming.

AlreadyStreaming: The camera is already streaming.

InvalidNode: The device Node is invalid.

CameraNotOpened: The camera has not been opened.

InvalidDeviceIndex: The input device index is invalid.

NoDeviceConnected: There is no depth camera connected or the camera has not been connected correctly.

NoPropertyValueGet: Cannot get the value for the specified property.

NoPropertyValueSet: Cannot set the value for the specified property.

SystemCallFail: System call API failed.

InvalidValue: One or more of the parameter values provided are invalid.

HidWriteFail: Cannot write the buffer to the hid handle.

HidReadFail: Cannot read the buffer from the hid handle.

UVCOpenFail: Open UVC Failed.

TimeoutError: Read on file descriptor timed out.

InvalidBuffer: Frame from the camera is invalid.

RGB_DCalibNotFound: RGB-D Calibration Data not found in Camera.

FrameSeparationFailed: Frame separation Failed.

RegisterCallBackFailed: Registering frame call back Failed.

CameraAlreadyOpen: Camera is already open.

InvalidDataMode: Invalid Data Mode for intended change.

Failed: Operation Failed.

ProcessPtrOpenFailed: Loading Dust Noise filter failed.

ProcessPtrCloseFailed: Closing Dust Noise filter failed.

NoGraphicCardFound: Graphics card required for Dust noise filter not available.

InvalidDepthMode: Invalid Depth Mode for intended change.

CameraNotSupported: Device does not support requested control.

SizeQueryFailed: Failed to query the size of the current frame.

Others: An unknown error occurred.

FilterNotApplied: Processing filter not applied.

TOFCalibrationNotFound: TOF calibration data not found in camera module.

DeviceNotCapable: Device does not support certain feature.

Others: An unknown error occurred.

AvgRegion

Contains two values that tells whether the Average of Depth and IR is calculated at the center of the scene or along the mouse pointer.

```
typedef enum {  
    Center = 0,  
    CustomPtr = 1,  
    Exit = -1,  
}AvgRegion;
```

Where,

Center: Tells that the average depth and IR is calculated at the center of the scene.

CustomPtr: Tells that the average depth and IR is calculated along the mouse pointer.

Exit: Tells to stop average depth and IR calculation.

UVCPropID

UVC controls for DepthVista is listed in UVCPropID enum.

```
typedef enum {  
    TOF_UVC_CID_BRIGHTNESS,  
    TOF_UVC_CID_CONTRAST,  
    TOF_UVC_CID_SATURATION,  
    TOF_UVC_CID_WB_AUTO,  
    TOF_UVC_CID_GAMMA,  
    TOF_UVC_CID_GAIN,  
    TOF_UVC_CID_PWR_LINE_FREQ,  
    TOF_UVC_CID_WB_TEMP,  
    TOF_UVC_CID_SHARPNESS,  
    TOF_UVC_CID_EXPOSURE_AUTO,  
    TOF_UVC_CID_EXPSOURE_ABS,  
}UVCPropID;
```

Where,

TOF_UVC_CID_BRIGHTNESS : Brightness Control ID.

TOF_UVC_CID_CONTRAST : Contrast Control ID.

TOF_UVC_CID_SATURATION : Saturation Control ID.

TOF_UVC_CID_WB_AUTO : Auto White balance Control ID.

TOF_UVC_CID_GAMMA : Gamma Control ID.

TOF_UVC_CID_GAIN : Gain Control ID.

TOF_UVC_CID_PWR_LINE_FREQ : Powerline Frequency Control ID.

TOF_UVC_CID_WB_TEMP : White balance Control ID.

TOF_UVC_CID_SHARPNESS : Sharpness Control ID.

TOF_UVC_CID_EXPOSURE_AUTO : Auto Exposure Control ID.

TOF_UVC_CID_EXPSOURE_ABS : Absolute Exposure Control ID.

CalibValidFlag

CalibValidFlag enum Contain flag values to specify about the board's Calibration status.

```
typedef enum {  
    Not_Calibrated = -1,  
    Only_VGA = 0,  
    Only_HD = 1,  
    Both_VGA_HD = 2  
}CalibValidFlag;
```

Where,

Not_Calibrated: Camera module is not calibrated.

Only_VGA: Camera module is calibrated only for VGA mode.

Only_HD: Camera module is calibrated only for HD mode.

Both_VGA_HD: Camera module is calibrated for both VGA and HD modes.

PixFormat

PixFormat enum Contain the Pixel format of the current frame.

```
typedef enum {  
    PIX_FMT_UYVY,
```



```
PIX_FMT_Y16,  
PIX_FMT_RGB  
}PixelFormat;
```

Where,

PIX_FMT_UYVY: UYVY format (All RGB frames)

PIX_FMT_Y16: Y16 format (Raw Depth, Confidence and IR frames)

PIX_FMT_RGB: RGB format (DepthColor map)

DeviceType

DeviceType enum Contain the Device types supported by the SDK.

```
typedef enum{  
    USB_IRD = 1,  
    GMSL_USB = 2,  
    CMOS_USB_IRD = 3,  
    ITOF_USB = 4,  
    ITOF_GMSL = 5,  
    RGBD_IRD = 6,  
} DeviceType
```

Where,

USB_IRD: Represents DepthVista USB-IRD variant

GMSL_USB: Represents DepthVista GMSL-USB variant

CMOS_USB_IRD: Represents DepthVista CMOS-USB-IRD variant

ITOF_USB: Represents DepthVista_Helix_USB_IRD

ITOF_GMSL: Represents DepthVista_Helix_GMSL_IRD

RGBD_IRD: Represents DepthVista RGBD-IRD variant

Structure

The details regarding Structures used will be discussed below.

DeviceInfo

Description: This structure contains the information of a device that is connected to the host PC. It holds device name, VID, PID, device path and the serial number of the camera.

Members:

```
typedef struct
{
    char deviceName[50];
    char vid[5];
    char pid[5];
    char devicePath[500];
    char serialNo[50];
    DeviceType devType;
}DeviceInfo;
```

Member Description:

- **deviceName:** This member holds the deviceName of the camera.
- **pid:** This member holds the Product ID specific for DepthVista.
- **vid:** This member holds the Vendor ID specific for e-con Systems.
- **devicePath:** This member holds the path in which the device is enumerated.
- **serialNo:** This holds the serial number of the device.
- **devType:** This holds the device type of the device.

IntrinsicCalibParams

Description: This structure contains the intrinsic calibration values of a device that is connected to the host PC. It holds fx, fy, cx, cy, k1, k2, p1, p2, k3 values of the camera.

Members:

```
typedef struct
{
    double fx, fy, cx, cy;
    double k1, k2, p1, p2, k3;
```

```
}IntrinsicCalibParams;
```

Member Description:

- **fx:** Focal length of the device in the x-direction.
- **fy:** Focal length of the device in the y-direction.
- **cx:** x-coordinate of the principal point.
- **cy:** y-coordinate of the principal point.
- **k1:** Radial distortion coefficient.
- **k2:** Radial distortion coefficient.
- **p1:** Tangential distortion coefficient.
- **p2:** Tangential distortion coefficient.
- **k3:** Radial distortion coefficient.

ExtrinsicCalibParams

Description: This structure contains the extrinsic calibration values of a device that is connected to the host PC. It holds rotationalVec, translationalVec vectors of the camera.

Members:

```
typedef struct
{
    double rotationalVec[3][3];
    double translationalVec[3];
}ExtrinsicCalibParams;
```

Member Description:

- **rotationalVec:** Rotational Matrix of the device.
- **translationalVec:** Translational Vector of the device.

CalibrationParams

Description: This structure contains all the calibration values of the device. It holds depthCamVGAIntrinsic, rgbCamVGAIntrinsic, depthCamHDItrinsic, rgbCamHDItrinsic, VGAextrinsic, HDextrinsic.

Members:

```
typedef struct
{
    IntrinsicCalibParams depthCamVGAIntrinsic,
    rgbCamVGAIntrinsic, depthCamHDItrinsic,
    rgbCamHDItrinsic;

    ExtrinsicCalibParams VGAextrinsic, HDextrinsic;
```

```
}calibrationParams;
```

Member Description:

- **depthCamVGAIntrinsic:** IntrinsicCalibParams structure which contains Intrinsic calibration values of depth camera in VGA mode.
- **rgbCamVGAIntrinsic:** IntrinsicCalibParams structure which contains Intrinsic calibration values of RGB camera in VGA mode.
- **depthCamHDItrinsic:** IntrinsicCalibParams structure which contains Intrinsic calibration values of depth camera in HD mode.
- **rgbCamHDItrinsic:** IntrinsicCalibParams structure which contains Intrinsic calibration values of RGB camera in HD mode.
- **VGAextrinsic:** ExtrinsicCalibParams structure which contains Extrinsic calibration matrix values in VGA mode.
- **HDextrinsic:** IntrinsicCalibParams structure which contains Extrinsic calibration matrix values in HD mode.

DeviceHandle

Description: This structure contains the DeviceHandle of a device that is connected to the host PC. It holds serialNo of the camera. This structure is used to differentiate multiple device.

Members:

```
typedef struct
{
    char serialNo[50];
}DeviceHandle;
```

Member Description:

- **serialNo:** Serial Number of the device

UVCProp

Description: This structure contains the details of the UVC properties queried from the device. It holds the UVC Property ID, minimum, maximum, current, step and the default value of a particular UVC property.

Members:

```
typedef struct
{
    int id;
    int min;
    int max;
```

```
    int cur;  
    int step;  
    int default_val;  
}UVCProp;
```

Member Description:

- **int id:** UVC Property ID.
- **int min:** Minimum value of that particular UVC property.
- **int max:** Maximum value of that particular UVC property.
- **int cur:** Current value of that particular UVC property.
- **int step:** Step value of that particular UVC property.
- **int default_val:** Default value of that particular UVC property.

DepthPtr

Description: This structure contains the x co-ordinate and y co-ordinate of the depth position.

Members:

```
typedef struct  
{  
    int X;  
    int Y;  
}DepthPtr;
```

Member Description:

- **X:** This member holds the x co-ordinate of the depth position.
- **Y:** This member holds the y co-ordinate of the depth position.

ToFFrame

Description: This structure contains the details of the frame obtained from camera. It holds the frame buffer, width, height, and size of the frame along with the pixel format.

Members:

```
typedef struct  
{  
    unsigned char* frame_data;  
    uint16_t width;  
    uint16_t height;  
    uint8_t pixel_format;
```

```
uint32_t total_size;
uint64_t time_stamp;
uint64_t frame_id;
}ToFFrame;
```

Member Description:

- **unsigned char* frame_data:** This unsigned char pointer holds the address of the memory that holds the frame data.
- **uint16_t width:** This holds the width of the frame.
- **uint16_t height:** This holds the height of the frame.
- **uint8_t pixel_format:** 0 for UYVY, 1 for Y16 and 2 for RGB pixel format.
- **uint32_t size:** This holds the size of the image buffer in bytes.
- **uint64_t time_stamp:** This holds the time stamp of the frame.
- **uint64_t frame_id:** This holds the id of the frame.

Frames

Description: This structure contains TofFrame of all frame types. It holds rgb, ir, raw_depth, depth_colormap.

Members:

```
typedef struct
{
    TofFrame rgb, ir, raw_depth, raw_depth_original,
    depth, depth_colormap, confidence_frame,
    confidence_frame2;
}Frames;
```

Member Description:

- **rgb:** TofFrame structure which contains frame data of rgb frame.
- **ir:** TofFrame structure which contains frame data of ir frame.
- **raw_depth:** TofFrame structure which contains frame data of depth frame without any post processing.
- **depth:** TofFrame structure which contains frame data of depth frame after post processing.
- **depth_colormap:** TofFrame structure which contains frame data of depth colormap frame.
- **confidence_frame:** TofFrame structure which contains frame data of confidence frame.
- **confidence_frame2:** TofFrame structure which contains frame data of second confidence frame.

IMUCONFIG_TypeDef

Description: This structure contains the configuration details for IMU.

Members:

```
typedef struct
{
    uint8_t IMU_MODE;
    uint8_t ACC_AXIS_CONFIG;
    uint8_t ACC_SENSITIVITY_CONFIG;
    uint8_t GYRO_AXIS_CONFIG;
    uint8_t GYRO_SENSITIVITY_CONFIG;
    uint8_t IMU_ODR_CONFIG;
} IMUCONFIG_TypeDef;
```

Member Description:

1. **IMU_MODE** – The operating mode of the IMU.
 - IMU_ACC_GYRO_DISABLE (0x00) - Accelerometer and Gyroscope Disabled.
 - IMU_ACC_ENABLE (0x01) - Accelerometer Enabled and Gyroscope Disabled.
 - IMU_GYRO_ENABLE (0x02) - Accelerometer Disabled and Gyroscope Enabled.
 - IMU_ACC_GYRO_ENABLE (0x03) - Accelerometer and Gyroscope Enabled.
2. **ACC_AXIS_CONFIG** - The configuration for enabling/disabling an axis of Accelerometer, it is a bitmap. For example, to enable X axis alone use the code 0x01, to enable X and Z axis use 0x05 (0x01 | 0x04).
 - IMU_ACC_X_Y_Z_ENABLE (0x07) - All 3 axis (X, Y, Z) enabled.
 - IMU_ACC_X_ENABLE (0x01) - Enable X Axis.
 - IMU_ACC_Y_ENABLE (0x02) - Enable Y Axis.
 - IMU_ACC_Z_ENABLE (0x04) - Enable Z Axis.
3. **ACC_SENSITIVITY_CONFIG** - The configuration to set the sensitivity of the Accelerometer.
 - IMU_ACC_SENS_2G (0x00) - Set sensitivity of 2g.

- IMU_ACC_SENS_4G (0x02) - Set sensitivity of 4g.
 - IMU_ACC_SENS_8G (0x03) - Set sensitivity of 8g.
 - IMU_ACC_SENS_16G (0x01) - Set sensitivity of 16g.
4. **GYRO_AXIS_CONFIG** - The configuration for enabling/disabling an axis of Gyroscope, it is a bitmap. For example, to enable X axis alone use the code 0x01, to enable X and Z axis use 0x05 (0x01 | 0x04).
- IMU_GYRO_X_Y_Z_ENABLE (0x07) - All 3 axis (X, Y, Z) enabled.
 - IMU_GYRO_X_ENABLE (0x01) - Enable X Axis.
 - IMU_GYRO_Y_ENABLE (0x02) - Enable Y Axis.
 - IMU_GYRO_Z_ENABLE (0x04) - Enable Z Axis.
5. **GYRO_SENSITIVITY_CONFIG** - The configuration to set the sensitivity of the Gyroscope.
- IMU_GYRO_SENS_245DPS (0x00) - Set sensitivity of 245 degrees per second.
 - IMU_GYRO_SENS_500DPS (0x02) - Set sensitivity of 500 degrees per second.
 - IMU_GYRO_SENS_2000DPS (0x03) - Set sensitivity of 2000 degrees per second.
6. **IMU_ODR_CONFIG** - The configuration of Output Data Rate (ODR) of the IMU.
- IMU_ODR_12_5HZ (0x01) - ODR is 12.5 Hz
 - IMU_ODR_26HZ (0x02) - ODR is 26 Hz
 - IMU_ODR_52HZ (0x03) - ODR is 52 Hz
 - IMU_ODR_104HZ (0x04) - ODR is 104 Hz.
 - IMU_ODR_208HZ (0x05) - ODR is 208 Hz.
 - IMU_ODR_416HZ (0x06) - ODR is 416 Hz.
 - IMU_ODR_833HZ (0x07) – ODR is 833 Hz.
 - IMU_ODR_1666HZ (0x08) – ODR is 1666 Hz.

IMUDATAINPUT_TypeDef

Description: This structure contains the configuration details of IMU for Control IMU Capture.

Members:

```
typedef struct
{
    uint8_t imu_update_mode;
    uint16_t imu_num_of_values;
} IMUDATAINPUT_TypeDef;
```

Member Description:

1. **IMU_UPDATE_MODE** – The value update mode of the IMU.
 - IMU_CONT_UPDT_EN (0x01) - Continuous Update of IMU Values Enabled.
 - IMU_CONT_UPDT_DIS (0x02) - Continuous Update of IMU Values Disabled.
2. **IMU_NUM_OF_VALUES** – The number of values required, currently need to be set to one, other values support will be added later.

IMUDATAOUTPUT_TypeDef

Description: This structure contains the output data from IMU.

Members:

```
typedef struct
{
    uint16_t imu_value_id;
    double accX;
    double accY;
    double accZ;
    double gyroX;
    double gyroY;
    double gyroZ;
} IMUDATAOUTPUT_TypeDef;
```

Member Description:

- **IMU_VALUE_ID** – The ID for each set of values generated from the IMU, range 1-65535 with reload.

- **accX** – The accelerometer g value for the X axis.
 - **accY** – The accelerometer g value for the Y axis.
 - **accZ** – The accelerometer g value for the Z axis.
 - **gyroX**– The gyroscope DPS (Degree per second) value for the X axis.
 - **gyroY**– The gyroscope DPS value for the Y axis.
- gyroZ**– The gyroscope DPS value for the Z axis.

IMU_DATA_GMSL

Description: This structure contains the output data from IMU.

Members:

```
typedef struct _imu_data {  
#if __linux__  
    __u16 index;  
    __u16 size;  
#endif  
  
    int16_t accX;  
    int16_t accY;  
    int16_t accZ;  
    int16_t gyroX;  
    int16_t gyroY;  
    int16_t gyroZ;  
    uint16_t imu_temp;  
    int8_t acc_flag;  
    int8_t gyro_flag;  
    int8_t temp_flag;  
    int8_t config_flag;  
    int8_t imu_mode;  
    int8_t odr_config;  
    int8_t acc_sensitivity;  
    int8_t gyro_sensitivity;  
} IMU_DATA_GMSL;
```

Member Description:

accX – The accelerometer g value for the X axis.

accY – The accelerometer g value for the Y axis.

accZ – The accelerometer g value for the Z axis.

gyroX– The gyroscope DPS (Degree per second) value for the X axis.

gyroY– The gyroscope DPS value for the Y axis.

gyroZ– The gyroscope DPS value for the Z axis.

imu_temp- Temperature reading of the IMU

acc_flag- true if the accelerometer fields are updated, else false

gyro_flag- true if the gyroscope fields are updated, else false

temp_flag- true if the temperature data is updated, else false

config_flag- true if IMU configuration is successful, else false

imu_mode- IMU mode value for configuring IMU

odr_config- ODR value for IMU configuration

acc_sensitivity- Accelerometer sensitivity value for IMU configuration

gyro_sensitivity- Gyroscope sensitivity value for IMU configuration

1. **We have already installed OpenCV version in PC. Will it make conflict with the DepthVista SDK?**

Pre-installed OpenCV version will not make any conflict with DepthVistaSDK, because the libs and header files required by DepthVistaSDK are statically bound. This also allows the user to use any version of **OpenCV above 4.5.4**.

Support

Contact Us

If you need any support on DepthVista product, please contact us using the Live Chat option available on our website - <https://www.e-consystems.com/>

Creating a Ticket

If you need to create a ticket for any type of issue, please visit the ticketing page on our website - <https://www.e-consystems.com/create-ticket.asp>

RMA

To know about our Return Material Authorization (RMA) policy, please visit the RMA Policy page on our website - <https://www.e-consystems.com/RMA-Policy.asp>

General Product Warranty Terms

To know about our General Product Warranty Terms, please visit the General Warranty Terms page on our website - <https://www.e-consystems.com/warranty.asp>

Revision History

Rev	Date	Description	Author
1.0	12-September-2025	Initial Draft	Camera Products
1.1	12-Feb-2026	Added GetIMUValueGMSL API	Camera Products
1.2	25-March-2026	Added SetConfidenceMode and GetConfidenceMode API	Camera Products
1.3	28-May-2026	Added Read SerialNo, Set/Get Laser Safety, Laser Reset, Set/Get FPS Mode.	Camera Products.